

Objective

The objective is to demonstrate real-time stream processing by building a pipeline in Kafka that continuously counts the occurrences of words in text data.

Commands:

```
jeeva@myNewVm:~$ cd kafka_2.13-4.2.0/
jeeva@myNewVm:~/kafka_2.13-4.2.0$ bin/kafka-server-start.sh config/server.properties
```

Create Topics

You need two topics: one for input text lines, one for output word counts.

- **streams-plaintext-input** → where you'll send text lines.
- **streams-wordcount-output** → where the WordCount app writes results.

Input Topic:

```
bin/kafka-topics.sh --create \
--topic streams-plaintext-input \
--bootstrap-server localhost:9092 \
--partitions 1 \
--replication-factor 1
```

Output Topic:

```
bin/kafka-topics.sh --create \
--topic streams-wordcount-output \
--bootstrap-server localhost:9092 \
--partitions 1 \
--replication-factor 1
```

- bin/kafka-topics.sh → Kafka's CLI tool to manage topics.
- --create → tells Kafka to create a new topic.
- --topic streams-plaintext-input → the name of the topic where you'll send raw text lines.
- --topic streams-wordcount-output → where the WordCount app will write results (word counts).
- --bootstrap-server localhost:9092 → connects to the Kafka broker running locally on port 9092.
- --partitions 1 → creates the topic with one partition (all data goes into a single partition).
- --replication-factor 1 → only one replica (no redundancy, fine for a demo).

```
jeeva@myNewVm:~/kafka_2.13-4.2.0$ bin/kafka-topics.sh --create \
--topic streams-plaintext-input \
--bootstrap-server localhost:9092 \
--partitions 1 \
--replication-factor 1
Created topic streams-plaintext-input.
```

```
jeeva@myNewVm:~/kafka_2.13-4.2.0$ bin/kafka-topics.sh --create \
--topic streams-wordcount-output \
--bootstrap-server localhost:9092 \
--partitions 1 \
--replication-factor 1
Created topic streams-wordcount-output.
```

Produce Input Data

```
bin/kafka-console-producer.sh \
--topic streams-plaintext-input \
--bootstrap-server localhost:9092
```

- bin/kafka-console-producer.sh → Kafka's CLI producer tool.
- --topic streams-plaintext-input → sends messages into the input topic you created.
- --bootstrap-server localhost:9092 → connects to the local broker.
- Then type the input

```
jeeva@myNewVm:~/kafka_2.13-4.2.0$ bin/kafka-console-producer.sh \
--topic streams-plaintext-input \
--bootstrap-server localhost:9092
```

```
jeeva@myNewVm:~/kafka_2.13-4.2.0$ bin/kafka-console-producer.sh \
--topic streams-plaintext-input \
--bootstrap-server localhost:9092
>hi hello
>hi welcome
>hello welcome
>
```

Run the WordCount Application

- This application consumes from streams-plaintext-input.
- It splits lines into words, groups them, and counts occurrences.
- Results are written into streams-wordcount-output.

bin/kafka-run-class.sh

**org.apache.kafka.streams.examples.wordcount.WordCountDemo **

config/server.properties

- bin/kafka-run-class.sh → runs a Java class from the Kafka distribution.
- org.apache.kafka.streams.examples.wordcount.WordCountDemo → the demo class included with Kafka that implements the WordCount logic.
- config/server.properties → configuration file with broker details.

```
jeeva@myNewVm:~/kafka_2.13-4.2.0$ bin/kafka-run-class.sh \  
> org.apache.kafka.streams.examples.wordcount.WordCountDemo \  
> config/server.properties
```

Consumer Output

**bin/kafka-console-consumer.sh **

**--topic streams-wordcount-output **

**--from-beginning **

**--bootstrap-server localhost:9092 **

**--formatter-property print.key=true **

**--formatter-property value.deserializer = org.apache.kafka **

.common.serialization.LongDeserializer

- bin/kafka-console-consumer.sh → Kafka's CLI consumer tool.
- --topic streams-wordcount-output → reads messages from the output topic.
- --from-beginning → starts reading from the very beginning of the topic (not just new messages).
- --bootstrap-server localhost:9092 → connects to the local broker.
- --formatter-property print.key=true → prints the key (the word itself).
- --formatter-property value.deserializer = org.apache.kafka \ .common.serialization.LongDeserializer → interprets the value as a long integer (the word count).

```
jeeva@myNewVm:~/kafka_2.13-4.2.0$ bin/kafka-console-consumer.sh \  
--topic streams-wordcount-output \  
--from-beginning \  
--bootstrap-server localhost:9092 \  
--formatter-property print.key=true \  
--formatter-property value.deserializer=org.apache.kafka.common.serialization.LongDeserializer  
  
jeeva@myNewVm:~/kafka_2.13-4.2.0$ bin/kafka-console-consumer.sh \  
--topic streams-wordcount-output \  
--from-beginning \  
--bootstrap-server localhost:9092 \  
--formatter-property print.key=true \  
--formatter-property value.deserializer=org.apache.kafka.common.serialization.LongDeserializer  
hi      1  
hello   1  
hi      2  
welcome 1  
hello   2  
welcome 2
```